> A complete deep-dive into the modern solution to Callback Hell — Promise lifecycle, consuming with .then()/.catch(), error handling, async/await syntax, and creating your own Promises.

Part1 — AJAX & XMLHttpRequest

AJAX (Asynchronous JavaScript And XML) is a technique to fetch data from a server without reloading the page.

XMLHttpRequest (XHR) is the original browser API for making HTTP requests.

How XHR Works

```
function getCountry(country) {
  const request = new XMLHttpRequest();

  // 1. Configure the request (method + URL)
  request.open("GET",
`https://restcountries.com/v3.1/name/${country}`);

  // 2. Send the request (async — doesn't block)
  request.send();

  // 3. Listen for when the response arrives
  request.addEventListener("load", function() {
    if (request.status === 200) {
      // 4. Parse the JSON response
      const [countryData] = JSON.parse(request.responseText);
      renderCountry(countryData);
    }
  });
}

getCountry("Egypt");
```

The XHR Lifecycle

```
1. new XMLHttpRequest()    →  Create the request object
2. .open(method, url)      →  Configure it (doesn't send yet)
3. .send()                 →  Actually send the request
4. "load" event fires      →  Response arrived
5. .status                 →  HTTP status code (200 = OK, 404 = Not
Found)
6. .responseText           →  Raw JSON string response
7. JSON.parse()            →  Convert string to JS object
```

Chaining Requests (The Problem)

The country app needs two requests: first get the country, then use its coordinates to get the weather:

```
function getCountry(country) {
  const request = new XMLHttpRequest();
  request.open("GET",
`https://restcountries.com/v3.1/name/${country}`);
  request.send();

  request.addEventListener("load", function() {
    if (request.status === 200) {
      const [countryData] = JSON.parse(request.responseText);
      renderCountry(countryData);

      // Second request - depends on first request's data
      const [lat, lng] = countryData.capitalInfo.latlng;
      const request2 = new XMLHttpRequest();
      request2.open("GET", `https://api.open-meteo.com/v1/forecast?
latitude=${lat}&longitude=${lng}&current_weather=true`);
      request2.send();

      request2.addEventListener("load", function() {
        if (request2.status === 200) {
          const capitalWeather = JSON.parse(request2.responseText);
          renderCapital(capitalWeather);
        }
      });
    }
```

```
  });
}
```

This works — but what if you needed a third, fourth, fifth dependent request?

Part 2 — Callback Hell

Callback Hell (also called the "Pyramid of Doom") is what happens when you nest many async operations inside each other:

```
setTimeout(function() {
  console.log("1 second");
  setTimeout(function() {
    console.log("2 seconds");
    setTimeout(function() {
      console.log("3 seconds");
      setTimeout(function() {
        console.log("4 seconds");
        // ... and it keeps going deeper
      }, 1000);
    }, 1000);
  }, 1000);
}, 1000);
```

Why Callback Hell is a Problem

```
Real world nested XHR callback hell:

getUser(userId, function(user) {
  getOrders(user.id, function(orders) {
    getOrderDetails(orders[0].id, function(details) {
      getPaymentInfo(details.paymentId, function(payment) {
        getInvoice(payment.invoiceId, function(invoice) {
          // We're 5 levels deep — hard to read, maintain, debug
        });
      });
    });
```

```
  });
});
```

Problems with Callback Hell:

1. **Readability** — code flows diagonally, not top-to-bottom
2. **Error handling** — you need to handle errors at every level
3. **Maintainability** — adding or reordering steps requires restructuring
4. **Debugging** — stack traces are confusing and unhelpful
5. **Reusability** — logic is buried inside closures

Part 3 — The Modern Solution: `fetch()` and Promises

`fetch()` is the modern replacement for XHR. It returns a **Promise** — an object that represents a future value.

```
const result = fetch('https://restcountries.com/v3.1/name/egypt');
console.log(result); // Promise {<pending>}
```

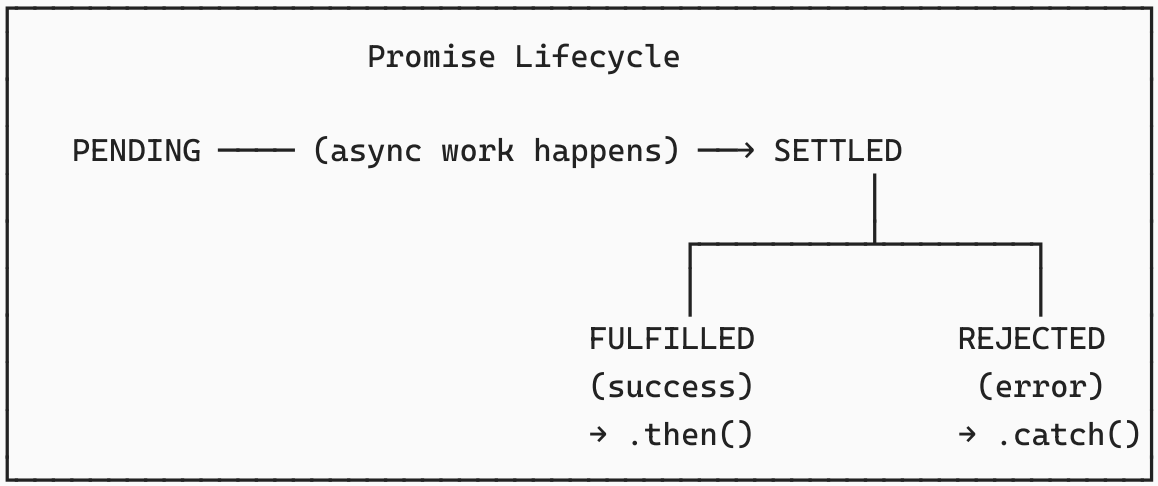
The response isn't available yet — it's a Promise in a "pending" state. This is the foundation for the next topics: Promises and async/await.

XHR vs fetch() — Key Differences

	XMLHttpRequest	fetch()
API style	Event-based callbacks	Promise-based
Readability	Verbose, nested	Clean, chainable
Error handling	Manual status checks	`.catch()` / try-catch
Modern?	Old (but widely supported)	Modern standard
Returns	Nothing (uses events)	Promise

Part 4 — The Promise Lifecycle

A Promise is an object that represents the **eventual result** of an async operation. It always exists in one of three states:



- **Pending** — the async operation is still in progress
- **Fulfilled** — it succeeded, a value is available
- **Rejected** — it failed, a reason (error) is available
- **Settled** — either fulfilled or rejected (not pending anymore)

> A Promise can only settle **once** — it's immutable after that. You can't go from fulfilled back to pending, or change the result.

Part 5 — Consuming Promises with `.then()` / `.catch()`

The Basic Pattern

```
fetch('https://restcountries.com/v3.1/name/egypt')
  .then(function(res) {
    return res.json(); // res.json() also returns a Promise!
  })
  .then(function([country]) {
    renderCountry(country);
  });
```

Why Two `.then()` Calls?

`fetch()` returns a Promise that resolves to a **Response object** — not the data yet. The Response contains the raw HTTP response. You must call `.json()` to parse the body, and `.json()` itself returns **another Promise**:

```
fetch(url)
  → Promise<Response>          (first .then)
    → res.json()
      → Promise<ParsedData>    (second .then — actual data)
```

Promise Chaining — The Solution to Callback Hell

The key insight: when you `return` a Promise inside a `.then()`, the next `.then()` waits for that Promise to settle(يستقر). This creates a **flat chain** instead of nested pyramids:

```
// FLAT chain — readable top to bottom
fetch('https://restcountries.com/v3.1/name/egypt')
  .then(res => res.json())
  .then(([country]) => {
    renderCountry(country);
    const [lat, lng] = country.capitalInfo.latlng;
    // Return a new Promise — the chain waits for it
    return fetch(`https://api.open-meteo.com/v1/forecast?
latitude=${lat}&longitude=${lng}&current_weather=true`);
  })
  .then(res => res.json())
  .then(data => renderCapital(data));
```

Compare this to the XHR callback hell version:

- Callbacks: nested inside each other, grows rightward
- Promises: chained, grows downward — linear and readable

> **Critical rule:** Always `return` a Promise from inside `.then()` if you want the chain to wait for it. Forgetting `return` breaks the chain — the next `.then()` runs

> immediately with `undefined`.

Error Handling with `.catch()`

`.catch()` catches any rejection that occurs **anywhere** in the chain above it:

```
function getCountry(country) {
  fetch(`https://restcountries.com/v3.1/name/${country}`)
    .then(res => res.json())
    .then(([country]) => {
      renderCountry(country);
      const [lat, lng] = country.capitalInfo.latlng;
      return fetch(`https://api.open-meteo.com/v1/forecast?...`);
    })
    .then(res => res.json())
    .then(data => renderCapital(data))
    .catch(err => console.log(err)); // catches errors from ANY step
above
}
```

The `fetch()` Trap — HTTP Errors Are NOT Rejected

`fetch()` only rejects on **network failures** (no internet, DNS failure). A `404 Not Found` or `500 Server Error` response still **fulfills** the Promise — it just has `res.ok === false`.

```
fetch(`https://restcountries.com/v3.1/name/xyzxyzxyz`)
  .then(res => {
    // res.ok is false for 4xx and 5xx status codes
    if (!res.ok) {
      throw new Error("Country Not found"); // manually reject
    }
    return res.json();
  })
  .then(data => renderCountry(data[0]))
  .catch(err => console.log(`${err.message}`));
```

> **Always check `res.ok`** when using fetch. Don't assume a fulfilled Promise means success.

Two-Argument `.then()` vs `.catch()`

`.then()` accepts two callbacks: one for success, one for failure:

```
// Two-argument form — handles error at THIS specific step only
fetch(url)
  .then(
    res => res.json(),          // success handler
    err => console.log(err)     // error handler (only for this step)
  )
  .then(([country]) => { ... });
```

vs.

```
// .catch() — handles ALL errors anywhere in the chain
fetch(url)
  .then(res => res.json())
  .then(([country]) => { ... })
  .catch(err => console.log(err)); // catches everything above
```

> Prefer `.catch()` at the end of the chain. It's cleaner and catches errors from all steps.

`.finally()` — Always Runs

`.finally()` runs whether the Promise fulfilled or rejected — useful for cleanup:

```
fetch(url)
  .then(res => res.json())
  .then(data => renderCountry(data[0]))
  .catch(err => console.log(err))
  .finally(() => {
    // Always runs — hide loading spinner, unlock UI, etc.
```

```
    spinner.style.display = "none";
  });
```

Part 6 — async / await

`async/await` is **syntactic sugar** over Promises — it makes async code look and behave like synchronous code, while still being non-blocking under the hood.

The `async` Keyword

Adding `async` before a function makes it automatically return a Promise:

```
async function getCountry(country) {
  // ... always returns a Promise
}

// Equivalent to:
function getCountry(country) {
  return new Promise((resolve, reject) => { ... });
}
```

The `await` Keyword

`await` pauses execution **inside** the async function until a Promise settles, then returns its resolved value:

```
// Without await — you get a Promise, not the data
const res = fetch(url); // Promise<Response>

// With await — you get the actual value
const res = await fetch(url); // Response object
```

> `await` can only be used **inside** an `async` function (or at the top level of ES modules).

Converting `.then()` Chain to async/await

```
// Promise chain version
function getCountry(country) {
  fetch(`https://restcountries.com/v3.1/name/${country}`)
    .then(res => res.json())
    .then(([data]) => {
      renderCountry(data);
      return fetch(`https://api.open-meteo.com/...`);
    })
    .then(res => res.json())
    .then(data => renderCapital(data))
    .catch(err => console.log(err));
}

// async/await version — reads like synchronous code
async function getCountry(country) {
  try {
    const res = await
fetch(`https://restcountries.com/v3.1/name/${country}`);
    if (!res.ok) throw new Error("Country Not found");

    const [data] = await res.json();
    renderCountry(data);

    const [lat, lng] = data.capitalInfo.latlng;

    const res2 = await fetch(`https://api.open-meteo.com/v1/forecast?
latitude=${lat}&longitude=${lng}&current_weather=true`);
    if (!res2.ok) throw new Error("Capital Not found");

    const weatherData = await res2.json();
    renderCapital(weatherData);

  } catch (err) {
    console.log(`${err.message}`);
  }
}

getCountry("Egypt");
```

Error Handling with try/catch

With async/await, use `try/catch` instead of `.catch()`:

```
async function example() {
  try {
    const data = await someAsyncOperation();
    // use data
  } catch (err) {
    // handles any error thrown inside the try block
    console.log(err.message);
  } finally {
    // always runs
  }
}
```

---

async/await Does NOT Block the Main Thread

Despite looking synchronous, `await` only pauses **inside the async function** — the rest of the program keeps running:

```
console.log("First");

async function getCountry(country) {
  const res = await
fetch(`https://restcountries.com/v3.1/name/${country}`);
  const [data] = await res.json();
  renderCountry(data);
}

getCountry("Egypt"); // starts the async work, doesn't block

console.log("After getCountry"); // runs immediately, before fetch
completes
```

Output:

```
First
After getCountry
(... country renders later when fetch completes)
```

Top-Level `await` (ES Modules)

In modern ES modules (`.mjs` or `type="module"`), you can use `await` at the top level without wrapping in an async function:

```
// In a module file
const res = await fetch('https://restcountries.com/v3.1/name/egypt');
const [country] = await res.json();
console.log(country.name.common);
```

Part 7 — Creating Your Own Promises

You create a Promise using `new Promise()` with an **executor function** that receives two callbacks: `resolve` and `reject` .

```
const promise = new Promise(function(resolve, reject) {
  // Do async work here
  setTimeout(function() {
    const success = true;

    if (success) {
      resolve("it worked!"); // fulfills the Promise, value passed to
.then()
    } else {
      reject("it failed!");  // rejects the Promise, reason passed to
.catch()
    }
  }, 1000);
});

promise
```

```
  .then(value => console.log(value)) // "it worked!"
  .catch(err => console.log(err));
```

What `resolve` and `reject` Actually Do

```
// Conceptually:
resolve = function(value) {
  changeStateTo("fulfilled");
  passValueToThen(value);     // .then(value => ...)
}

reject = function(reason) {
  changeStateTo("rejected");
  passValueToCatch(reason);   // .catch(reason => ...)
}
```

Building a Custom `myFetch()` — Wrapping XHR in a Promise

This is how libraries like `axios` work internally — wrap old callback-based APIs in Promises:

```
function myFetch(url) {
  return new Promise((resolve, reject) => {
    const request = new XMLHttpRequest();
    request.open("GET", url);
    request.send();

    request.onload = function() {
      if (request.status === 200) {
        resolve(JSON.parse(request.responseText));
      } else {
        reject(new Error(`HTTP Error: ${request.status}`));
      }
    };

    request.onerror = function() {
      reject(new Error("Network error"));
    };
```

```
  });
}

// Now usable with .then()/.catch() just like fetch()
myFetch('https://restcountries.com/v3.1/name/egypt')
  .then(([country]) => renderCountry(country))
  .catch(err => console.log(err));
```

Promise Chaining with `.add()` Analogy

```
// Set chaining (you already know this):
set.add(20).add(30).add(40);

// Promise chaining — same concept, each .then() returns a new
Promise:
fetch(url)
  .then( ... )
  .then( ... )
  .then( ... )
  .catch( ... );
```

Each method returns the object itself (or a new Promise), enabling the chain.

Part 8 — Promises vs async/await — When to Use Which

	`.then()` / `.catch()`	**async / await**
Readability	Good for simple chains	Better for complex logic
Error handling	`.catch()` at the end	`try/catch` block
Parallel requests	`.then()` with `Promise.all()`	`await Promise.all()`
Debugging	Stack traces can be tricky	Cleaner stack traces
In event handlers	Fine	Fine
Conditional logic	Gets messy with branches	Clean with if/else

> In practice, most code uses `async/await` for its readability. But understanding `.then()` is essential — async/await is just a syntax layer on top of Promises.

Part 9 — Complete Summary

```
Promise Lifecycle:
  pending → fulfilled (resolve) → .then(value)
  pending → rejected  (reject)  → .catch(reason)

Consuming Promises:
  .then(onSuccess)          – handle fulfilled value
  .catch(onError)           – handle any rejection in chain
  .finally(always)          – runs regardless of outcome
  Always check res.ok!      – fetch() doesn't reject on 4xx/5xx

Promise Chaining:
  Return a Promise inside .then() to chain async steps
  Flat chain = no more callback hell

async/await:
  async fn()    – function always returns a Promise
  await promise – pause inside async fn, get resolved value
  try/catch     – handle errors (replaces .catch())
  Looks sync, IS async – doesn't block the main thread

Creating Promises:
  new Promise((resolve, reject) => { ... })
  resolve(value)  – fulfill the promise
  reject(reason)  – reject the promise
  Use to wrap old callback APIs in Promise interface
```

Abdullah Ali

Contact : +201012613453